

TECHNICAL ARTICLE

Your PostgreSQL Index Is Used - Why Is the Query Still Slow?

*Selectivity, Heap Access, Planner Statistics, Execution Plans,
Troubleshooting, and Fixes*

Validated against PostgreSQL 18 official documentation

Prepared as an operations-focused performance reference | September 2026

Scope Explains why index usage is not the same as index benefit. Includes a reproducible lab, plan-reading method, statistics diagnostics, production troubleshooting, safe fixes, and corrected technical notes. Timings are examples, not universal benchmarks.

Table of Contents

1. Executive Summary	3
2. What an Index Really Does	3
3. Reproducible Lab Setup	4
4. Planner Statistics and Selectivity	4
5. Index Scan vs. Bitmap vs. Sequential Scan	5
6. Reading the Execution Plan Correctly	6
7. Why an Index Can Be Used but Still Be Slow	7
8. Step-by-Step Troubleshooting Workflow	7
9. Fix Patterns and SQL Examples	8
10. Monitoring and Diagnostic Scripts	10
11. Common Mistakes and Corrections	10
12. Production Checklist	11
13. Official-Documentation Validation	12
14. References	12

1. Executive Summary

An execution plan that contains Index Scan, Bitmap Index Scan, or Index Cond proves only that PostgreSQL chose an index access path. It does not prove that the plan is efficient. A query can use an index and still read a large share of the heap, return too many rows, perform expensive sorts or joins, wait on I/O, or spend time fetching wide tuples.

Key conclusion Measure benefit by work avoided and time spent - not by the presence of an index node. Use EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS) where safe, compare estimated and actual rows, inspect heap blocks and loops, and account for cache state and concurrency.

The source example correctly demonstrates the role of selectivity, but its exact 700x ratio is environment-specific. The claim that ANALYZE after changing five rows is always required and will reliably teach the planner about a five-row rare value needs correction: ANALYZE samples large tables, so five rows can be missed. A partial index, a more selective predicate, higher statistics target, or application-aware design may be more reliable depending on the requirement.

2. What an Index Really Does

A B-tree stores ordered key values with tuple identifiers that locate heap tuples. A normal Index Scan follows matching index entries to the heap. A Bitmap Index Scan first creates a bitmap of matching tuple locations; the Bitmap Heap Scan then visits heap pages in physical order. An Index-Only Scan can avoid many heap visits only when all requested columns are available from the index and visibility-map information proves that heap visibility checks are unnecessary.

Access path	Best fit	Main cost risk
Index Scan	Few matching rows; ordered output can be useful.	Many random or poorly clustered heap fetches.
Bitmap Heap Scan	Moderate number of matches spread across pages.	Most heap pages still visited; bitmap may become lossy under memory pressure.
Sequential Scan	Large fraction of table or small table.	Reads every table page, but efficient sequential access can be cheapest.
Index-Only Scan	Narrow projection covered by index and many all-visible pages.	Heap fetches remain when visibility map cannot confirm visibility.

Important Creating a normal index does not reorder the heap. CLUSTER can physically reorder a table once, but later writes can reduce that ordering and CLUSTER requires operational planning.

3. Reproducible Lab Setup

```
DROP TABLE IF EXISTS users;
CREATE TABLE users (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  last_name varchar(40),
  first_name varchar(40),
  country char(2),
  created_at timestamp NOT NULL DEFAULT clock_timestamp(),
  updated_at timestamp NOT NULL DEFAULT clock_timestamp()
);

INSERT INTO users(last_name, first_name, country)
SELECT 'Last' || floor(random()*5000)::int,
  (ARRAY['Alice','Bob','David','Sara','Mike','Sam','Joe','Peter',
    'Rahul','Hassan','Adam','Ali','Fabio','Flora','John',
    'Nial','Pamola','Pedro','Perao','Saira'])[floor(random()*20)+1],
  (ARRAY['US','UK','CA','AU','IE'])[floor(random()*5)+1]
FROM generate_series(1,500000);

CREATE INDEX users_first_name_idx ON users(first_name);
ANALYZE users;
```

Use a disposable test database. `random()` makes the distribution and physical page placement differ on every run, so record your actual row counts, table size, cache condition, PostgreSQL version, and settings.

```
SELECT count(*) AS rows,
  pg_size_pretty(pg_relation_size('users')) AS heap,
  pg_size_pretty(pg_relation_size('users_first_name_idx')) AS index;

SELECT first_name, count(*)
FROM users GROUP BY first_name ORDER BY count(*) DESC;
```

4. Planner Statistics and Selectivity

4.1 What `pg_stats` means

```
SELECT attname, null_frac, n_distinct, avg_width,
  most_common_vals, most_common_freqs, correlation
FROM pg_stats
WHERE schemaname = 'public' AND tablename = 'users'
ORDER BY attname;
```

Statistic	Meaning	Performance relevance
<code>n_distinct > 0</code>	Estimated fixed number of distinct values.	Equality estimates divide non-MCV population across remaining values.

Statistic	Meaning	Performance relevance
n_distinct < 0	Negative fraction of rows; -1 means approximately unique.	Distinct count is expected to scale with table size.
most_common_vals / freqs	Sample-derived frequent values and their table fractions.	Directly influences equality selectivity for listed values.
histogram_bounds	Approximate distribution excluding MCV values.	Used mainly for range selectivity.
correlation	Relationship between logical key order and physical heap order.	Near +/-1 can reduce estimated index heap-fetch cost.

4.2 Statistics are approximate

PostgreSQL ANALYZE samples large tables. Fresh statistics are still estimates and can vary between runs. A value present in only five of 500,000 rows is unlikely to be captured as a most-common value by the default sample. Therefore, manual ANALYZE after five changes is not a guarantee that the planner will estimate five rows.

```
ALTER TABLE users ALTER COLUMN first_name SET STATISTICS 500;
ANALYZE users (first_name);

SELECT attname, n_distinct, most_common_vals, most_common_freqs
FROM pg_stats
WHERE schemaname='public' AND tablename='users'
AND attname='first_name';
```

Use with care A higher statistics target increases sample size, ANALYZE time, and catalog storage. It improves estimates for some skewed distributions but still does not make a tiny rare value certain to appear in the MCV list.

5. Index Scan vs. Bitmap vs. Sequential Scan

The planner compares estimated costs. It considers matching-row estimates, table and index pages, physical correlation, cache assumptions, CPU costs, and storage cost parameters. There is no universal selectivity percentage at which an index stops being useful.

Factor	Favors index access	Favors sequential access
Rows matched	Very few	Large share of table

Factor	Favors index access	Favors sequential access
Heap page coverage	Few pages	Most pages
Correlation	Rows clustered by index order	Matches scattered randomly
Projection	Covered/narrow columns	SELECT * or wide rows
Cache and storage	Hot cache / low random latency	High random I/O relative to sequential I/O
Ordering / LIMIT	Index provides order and stops early	No useful index order or most rows needed

```
SET track_io_timing = on; -- superuser or platform permission may be required
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, SUMMARY)
SELECT * FROM users WHERE first_name = 'Alice';
```

A bitmap plan is not a failed index plan. It is a deliberate compromise: collect matching tuple locations through the index, sort access by heap page, then fetch rows. It can still be slow if the qualifying tuples are distributed across almost all heap pages.

6. Reading the Execution Plan Correctly

Plan field	Question to ask
cost / rows / width	What did the planner expect before execution?
actual time / rows / loops	What happened, and how many times did the node repeat?
Index Cond	Which condition navigated the index?
Filter / Rows Removed by Filter	How much work happened after access?
Heap Blocks exact/lossy	How many heap pages were represented precisely or approximately?
Buffers hit/read/dirtied/written	Was work served from cache, read from storage, or modified?
Heap Fetches	For Index-Only Scan, how often was heap visibility still checked?
I/O timings	Where supported, how much elapsed I/O time was recorded?

```
SELECT c.relname, c.reltuples::bigint AS estimated_rows,
       c.relpages AS heap_pages,
       pg_size_pretty(pg_relation_size(c.oid)) AS heap_size
FROM pg_class c
WHERE c.oid = 'public.users'::regclass;
```

Calculation Heap Blocks from a plan can be compared with `pg_class.relpages` as a rough page-

coverage indicator. Do not treat the ratio as exact when parallel workers, repeated loops, concurrent growth, partitioning, or lossy bitmaps are involved.

7. Why an Index Can Be Used but Still Be Slow

Cause	Plan evidence	Meaning
Low selectivity	Actual rows are a large share of table.	Index cannot skip much data.
Scattered matches	Heap blocks approach table pages; low correlation.	Many heap pages must still be fetched.
Stale/inadequate statistics	Large estimated-vs-actual row gap.	Planner priced the alternatives incorrectly.
Wide SELECT *	Large width and many heap fetches.	Index-only access is impossible; transfer and tuple deformation cost rise.
Residual filtering	Many rows removed by filter.	Index condition does not cover the full predicate.
Repeated nested-loop probe	High loops on inner index node.	Fast single lookup multiplied many times becomes expensive.
Lossy bitmap	Lossy heap blocks and rechecks.	work_mem was insufficient for an exact bitmap.
Cache/I/O effects	High shared reads and I/O time.	Same plan can be much slower cold or under storage pressure.
Bloat / dead tuples	Index entries read exceed live rows; large relation size.	More pages and visibility checks are required.
Locks or CPU/concurrency	Execution time high without matching buffer/I/O work.	The bottleneck may be outside access-path choice.

8. Step-by-Step Troubleshooting Workflow

1. Capture the exact SQL, parameters, result-row requirement, PostgreSQL version, and runtime context.

2. Use EXPLAIN without ANALYZE first for unsafe writes; use EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS) only when execution is safe.
3. Compare estimated rows with actual rows at every important node, including loops.
4. Measure returned rows as a share of the table and heap pages touched as a share of relpages.
5. Inspect Index Cond, Filter, Rows Removed by Filter, lossy blocks, heap fetches, sorts, joins, and temporary I/O.
6. Inspect pg_stats for MCVs, n_distinct, correlation, null fraction, and statistics target.
7. Check table/index size, bloat indicators, cache state, I/O timing, locks, CPU, and concurrency.
8. Test the smallest evidence-based change in a lower environment and compare plans with the same cache and data conditions.
9. Validate write overhead, index size, maintenance impact, and plan stability before production rollout.
10. Monitor after deployment and keep a rollback path.

9. Fix Patterns and SQL Examples

9.1 Return less data

```
SELECT id, first_name, country
FROM users
WHERE first_name = 'Alice'
ORDER BY id
LIMIT 100;
```

Pagination or a more selective business predicate often provides a larger improvement than adding another index to a query that must return tens of thousands of wide rows.

9.2 Create a composite index for the real predicate

```
CREATE INDEX CONCURRENTLY users_country_first_name_idx
ON users(country, first_name);

EXPLAIN (ANALYZE, BUFFERS)
SELECT id, first_name, country
FROM users
WHERE country='US' AND first_name='Alice';
```

Column order should match actual equality/range and ordering patterns. A composite index is not automatically better and adds write, storage, vacuum, and cache cost.

9.3 Cover a narrow query

```
CREATE INDEX CONCURRENTLY users_first_name_cover_idx
ON users(first_name) INCLUDE (id, country);

VACUUM (ANALYZE) users;
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, first_name, country FROM users WHERE first_name='Alice';
```


INCLUDE makes columns available to index-only scans but does not make them search keys. Index-only benefit depends on visibility-map coverage and workload churn.

9.4 Partial index for a stable rare condition

```
CREATE INDEX CONCURRENTLY users_rare_name_idx
ON users(first_name)
WHERE first_name = 'Zzrare';

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM users WHERE first_name='Zzrare';
```

A partial index is useful only when its predicate matches a stable business rule and the planner can prove the query implies that predicate. Parameterized query forms may limit implication at planning time.

9.5 Improve statistics only when estimates justify it

```
ALTER TABLE users ALTER COLUMN first_name SET STATISTICS 500;
ANALYZE users (first_name);

-- Reset to the inherited default if the higher target is not useful:
ALTER TABLE users ALTER COLUMN first_name SET STATISTICS -1;
```

9.6 Review cost settings instead of forcing scans

```
SHOW random_page_cost;
SHOW seq_page_cost;
SHOW effective_cache_size;
SHOW effective_io_concurrency;
SHOW work_mem;

SELECT name, setting, unit, source
FROM pg_settings
WHERE name IN ('random_page_cost', 'seq_page_cost', 'effective_cache_size',
               'effective_io_concurrency', 'work_mem');
```

Warning Do not use `enable_seqscan=off` or other `enable_*` switches as a permanent fix. They are diagnostic controls, and disabling a plan type only discourages it; it does not correct selectivity, schema, or cost-model problems.

10. Monitoring and Diagnostic Scripts

10.1 Index and table usage counters

```
SELECT s.schemaname, s.relname, s.indexrelname,
       s.idx_scan, s.idx_tup_read, s.idx_tup_fetch,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS index_size
FROM pg_stat_user_indexes s
WHERE s.relname='users'
ORDER BY s.idx_scan DESC;
```

10.2 Table scan and tuple activity

```
SELECT relname, seq_scan, seq_tup_read, idx_scan, idx_tup_fetch,
       n_live_tup, n_dead_tup, last_analyze, last_autoanalyze,
       last_vacuum, last_autovacuum
FROM pg_stat_user_tables
WHERE relname='users';
```

10.3 Physical correlation and statistics target

```
SELECT a.attname, a.attstattarget, s.n_distinct, s.correlation
FROM pg_attribute a
JOIN pg_stats s ON s.schemaname='public'
  AND s.tablename='users' AND s.attname=a.attname
WHERE a.attrelid='public.users'::regclass AND a.attnum > 0
ORDER BY a.attnum;
```

10.4 Safe plan capture pattern

```
-- Read-only query example:
BEGIN READ ONLY;
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, FORMAT TEXT)
SELECT id, first_name, country
FROM users WHERE first_name='Alice';
ROLLBACK;
```

EXPLAIN ANALYZE executes the statement. A read-only transaction protects only against writes; it does not make an expensive SELECT harmless. Use `statement_timeout`, a representative lower environment, and production change controls.

11. Common Mistakes and Corrections

Claim or mistake	Correct operational view
"The plan uses the index, so indexing is solved."	Inspect actual work, pages, rows, loops, filters, I/O, and total runtime.
"A 5% predicate should always use a bitmap."	No fixed threshold exists; distribution, row width, cache, correlation, cost settings, and LIMIT matter.
"Heap Blocks exact=4548 means 4548 disk reads."	It describes exact bitmap heap blocks, not necessarily physical reads. BUFFERS hit/read and I/O timing show cache/storage behavior.
"ANALYZE after five updates guarantees the rare value is known."	ANALYZE samples large tables; five rows may be absent from the sample and MCV list.
"No ANALYZE is needed after CREATE INDEX."	A plain index does not require fresh column statistics solely because it was created, but analyze may still be needed if the data distribution changed or expression statistics are required.

Claim or mistake	Correct operational view
“The same index made one query 700x faster.”	The observed ratio combines selectivity, returned rows, heap pages, cache, hardware, and measurement conditions; it is not an index property.
“SELECT * is fine because the predicate is indexed.”	Wide projection forces heap access and increases CPU, buffers, and network transfer.
“Force index scans to fix the planner.”	Fix evidence: statistics, query, index design, data layout, or calibrated costs; forcing plan types is fragile.

12. Production Checklist

- ☐ The actual plan was captured safely with BUFFERS and representative parameters.
- ☐ Estimated versus actual rows were reviewed at every major node.
- ☐ Heap page coverage and returned-row percentage were calculated.
- ☐ Cache state, shared reads, I/O time, CPU, locks, and concurrency were considered.
- ☐ pg_stats MCVs, n_distinct, correlation, and statistics target were reviewed.
- ☐ Index design matches the complete predicate, projection, ordering, and LIMIT.
- ☐ Write overhead, size, vacuum, and deployment impact were tested.
- ☐ CREATE INDEX CONCURRENTLY limitations and failure cleanup are understood.
- ☐ The change was validated with repeatable warm and cold tests where possible.
- ☐ Monitoring and rollback steps are documented.

13. Official-Documentation Validation

Source concept	Validation result
B-tree points from keys to heap tuples	Correct as a simplified model.
Bitmap Heap Scan visits pages containing matches	Correct; it visits heap pages in physical order and may use exact or lossy bitmap entries.
Selectivity determines whether an index helps	Correct but incomplete; row width, correlation, cache/storage, costs, ordering, LIMIT, and concurrency also matter.
n_distinct=-1 means unique and scales with rows	Correct.

Source concept	Validation result
MCV frequencies drive equality estimates	Correct for values represented in the MCV list.
Heap Blocks exact near relpages means the index skips little	Useful diagnostic, but it is not the same as physical reads and needs BUFFERS context.
ANALYZE is required after changing five rows	Not a universal rule; auto-analyze may not trigger, but manual ANALYZE sampling may also miss the rare rows.
Rare-value plan estimated rows=1 after ANALYZE	Possible in a particular run, not guaranteed by the shown data-generation method.
700x faster on the same index	Valid observation for that test only; not a portable expectation.

14. References

- PostgreSQL 18 - Indexes: <https://www.postgresql.org/docs/18/indexes.html>
- PostgreSQL 18 - Combining Multiple Indexes / Bitmap Scans: <https://www.postgresql.org/docs/18/indexes-bitmap-scans.html>
- PostgreSQL 18 - Index-Only Scans and Covering Indexes: <https://www.postgresql.org/docs/18/indexes-index-only-scans.html>
- PostgreSQL 18 - Using EXPLAIN: <https://www.postgresql.org/docs/18/using-explain.html>
- PostgreSQL 18 - EXPLAIN Command: <https://www.postgresql.org/docs/18/sql-explain.html>
- PostgreSQL 18 - Planner Statistics: <https://www.postgresql.org/docs/18/planner-stats.html>
- PostgreSQL 18 - Row Estimation Examples: <https://www.postgresql.org/docs/18/row-estimation-examples.html>
- PostgreSQL 18 - pg_stats: <https://www.postgresql.org/docs/18/view-pg-stats.html>
- PostgreSQL 18 - ANALYZE: <https://www.postgresql.org/docs/18/sql-analyze.html>
- PostgreSQL 18 - Planner Cost Constants: <https://www.postgresql.org/docs/18/runtime-config-query.html#RUNTIME-CONFIG-QUERY-CONSTANTS>
- Source article - Mudassar Hassan: <https://mudassarhassan.com/posts/postgres-index-used-not-helpful/>

Validation basis: PostgreSQL 18 official documentation was used for planner behavior, statistics, EXPLAIN interpretation, indexes, and ANALYZE. Confirm the documentation for the deployed major version before changing production settings.